DATA 37711: Homework 1

Question 1

Let a be the amount Victor bets, with $0 \le a \le d$ and $d > 0$. I use the market probability 0.60 to interpret the odds: a contract costs \$0.60 and pays \$1 if Biden wins. Each dollar bet therefore earns a net profit of $0.40/0.60 = 2/3$ dollars if he wins. Victor's final wealth is

$$W(a) = \begin{cases} d + \frac{2}{3}a, & \text{if Biden wins,} \\ d - a, & \text{if Biden loses.} \end{cases}$$

(a) If utility equals wealth, expected utility is

$$\begin{aligned} \mathbb{E}[U(W(a))] &= 0.85\left(d + \frac{2}{3}a\right) + 0.15(d - a) \\ &= d + \frac{5}{12}a. \end{aligned}$$

Since $5/12 > 0$, expected utility increases with the amount bet. Thus, the best choice is $\boxed{a^* = d}$.

The linear utility assumption posits that people are risk-neutral and only care about expected wealth. However, this may not make sense in real life, Since the cost of losing all of the money is very dangerous, and having one extra dollar is not so critical for wealthy people.

(b) With $U(W) = \log W$, expected utility is

$$g(a) = 0.85 \log\left(d + \frac{2}{3}a\right) + 0.15 \log(d - a), \qquad 0 \le a < d.$$

Betting everything would give zero wealth with probability 0.15, so expected log utility would be $-\infty$. Differentiate and set the derivative equal to zero:

$$\begin{aligned} g'(a) &= \frac{0.85(2/3)}{d + (2/3)a} - \frac{0.15}{d - a} = 0, \\ 0.85 \cdot \frac{2}{3}(d - a) &= 0.15\left(d + \frac{2}{3}a\right), \\ \frac{5}{12}d &= \frac{2}{3}a. \end{aligned}$$

Therefore, $\boxed{a^* = \frac{5}{8}d = 0.625d}$. Both log terms are strictly concave in a, so this feasible stationary point is the unique maximum.

(c) Personally, I will not make this bet.

\$500,000 is already a great amount of money, I may want to keep that and do some more stable investment. Part (b) suggests betting \$312,500, leaving only \$187,500 if I losewhich the result of losing the bet is so high. What's more, the high probability of winning (p=0.85) is only a subjective estimation, I would not like to take that risk.

Question 2

For binary Y, let $f(x)$ be the predicted probability that $Y = 1$. The cross-entropy loss is

$$L(Y, f(X)) = -Y \log f(X) - (1 - Y) \log(1 - f(X)).$$

Fix a value of x, and write

$$p = P(Y = 1 \mid X = x), \qquad q = f(x).$$

Since $Y = 1$ with probability p and $Y = 0$ with probability $1 - p$, the expected loss conditional on $X = x$ is

$$R_x(q) = -p \log q - (1 - p) \log(1 - q).$$

First assume $0 < p < 1$. For $0 < q < 1$, the derivative is

$$R'_x(q) = -\frac{p}{q} + \frac{1 - p}{1 - q} = \frac{q - p}{q(1 - q)}.$$

The denominator is positive. Thus, the derivative is negative when $q < p$ and positive when $q > p$. The expected loss decreases until $q = p$ and increases after it, so the unique minimum is $q = p$.

For the boundary cases, if $p = 0$, the expected loss is $-\log(1 - q)$ and is minimized at $q = 0$. If $p = 1$, it is $-\log q$ and is minimized at $q = 1$. Here the loss at a boundary is defined by its limit.

Finally, by the law of total expectation,

$$\mathbb{E}_{X,Y}[L(Y, f(X))] = \mathbb{E}_X[R_X(f(X))].$$

Choosing the conditional minimum for each x therefore minimizes the overall risk. Hence,

$$\boxed{f^*(x) = P(Y = 1 \mid X = x)}$$

for almost every x under the distribution of X. The probability of $Y = 0$ is then $1 - f^*(x)$. Thus, cross-entropy is a proper scoring rule.

Question 3

The training data is only a part of the overall dataset, so sampling errors is inevitable. Even if the precise minimum value of the training error is found, it does not necessarily guarantee better prediction performance on new data. Therefore, a small amount of optimization error can be accepted.

Also, a suitable small fixed learning rate can quickly reduce the training error. Although the parameters may fluctuate around the minimum value, as long as the fluctuations are small enough, the prediction can still be very good. With limited samples, we pay more attention to prediction performance and computational cost, so using a fixed learning rate and stopping training based on the performance of the validation set is reasonable.

Question 4

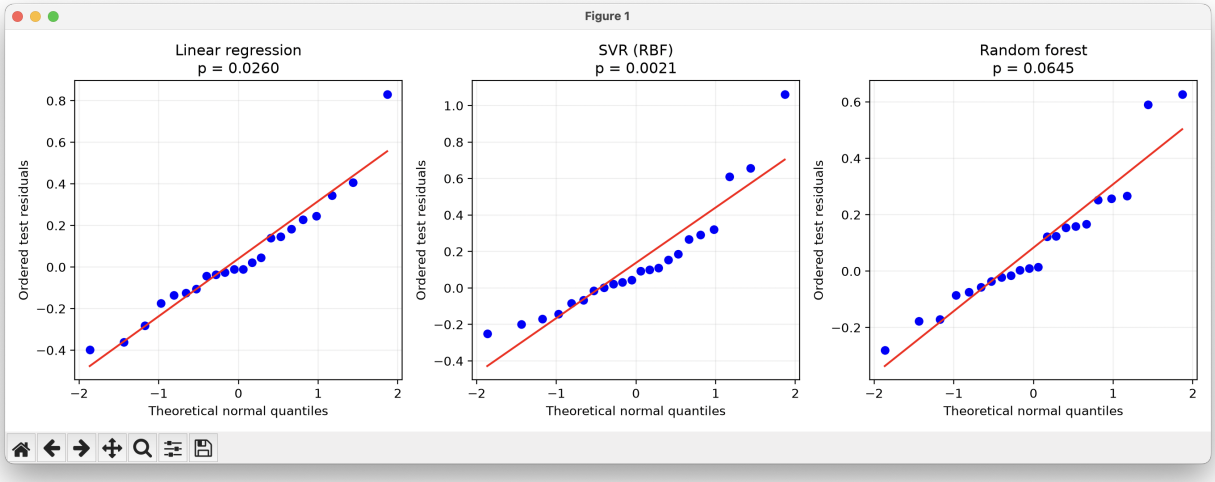
I excluded `sd_ale` and split the 107 observations into 85 training observations and 22 test observations using an 80/20 split with `random_state=42`. I fitted linear regression, support vector regression with an RBF kernel ($C = 1$, $\epsilon = 0.1$), and a random forest with 300 trees and `random_state=42`.

For each model, I calculated the test residuals as

$$e_i = y_i - \hat{y}_i.$$

I examined normal QQ plots and used `scipy.stats.normaltest`. The null hypothesis is that the residuals follow a normal distribution. I used a 5% significance level for each test.

Model	Test statistic	p-value	Decision
Linear regression	7.2997	0.0260	Reject normality
SVR (RBF)	12.3384	0.0021	Reject normality
Random forest	5.4819	0.0645	Do not reject



The normality test rejects the null hypothesis for linear regression and SVR because their p-values are below 0.05. For the random forest, the p-value is 0.0645, so I do not reject normality. However, this does not prove that its residuals are normal. There are only 22 test residuals, so the test has limited power.

In the QQ plot, the points close to the reference line indicate normality, while those deviating from the line suggest deviations from the normal distribution. Overall, the test results show that not all fitted models support normal residuals. However, this does not mean that the squared error loss is ineffective. Minimizing the expected squared error can still recover the conditional mean, and it does not require the use of normal residuals.

Code and running instructions

```
python3 -m venv .venv
.venv/bin/python -m pip install --upgrade pip
.venv/bin/python -m pip install numpy pandas scipy scikit-learn matplotlib
```

Question 5: Classification

Data and baseline

I split the Wisconsin Diagnostic Breast Cancer dataset into 80% training and 20% test data using `random_state=42` and stratification by label. This produced 455 training observations and 114 test observations.

For the baseline, I used scikit-learn's `LogisticRegression` with `C=1.0`, the `lbfgs` solver, and `max_iter=1000`. Its test accuracy was

$$\frac{112}{114} \approx 0.982456 = 98.25\%.$$

(a) Logistic regression using PyTorch

I implemented the model

$$p_i = \sigma(w^\top x_i + b), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

The training objective was the mean binary cross-entropy:

$$\mathcal{L}(w, b) = -\frac{1}{n} \sum_{i=1}^{n} \left[y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \right].$$

I initialized the weights and bias to zero and trained the model using `torch.optim.SGD`, with a learning rate of 0.01, a batch size of 32, and 500 epochs. The training data were shuffled each epoch. PyTorch computed the gradients automatically. I clipped probabilities to $[10^{-7}, 1 - 10^{-7}]$ before evaluating the logarithms.

For prediction, I assigned label 1 when the predicted probability was at least 0.5 and label 0 otherwise. The model correctly classified 111 of the 114 test observations:

$$\text{Test accuracy} = \frac{111}{114} \approx 0.973684 = 97.37\%.$$

This was about 0.88 percentage points below the baseline, corresponding to one fewer correct prediction.

The implementations also differ in their training objectives: the scikit-learn baseline uses its default L2 regularization, whereas my PyTorch model minimizes binary cross-entropy without an additional penalty.

(b) Probit regression using PyTorch

For probit regression, I replaced the sigmoid function with the standard normal cumulative distribution function:

$$p_i = \Phi(w^\top x_i + b).$$

I implemented Φ using `torch.distributions.Normal(0.0, 1.0).cdf`.

I used the same binary cross-entropy loss, probability clipping, initialization, SGD settings, and classification threshold as in part (a).

The probit model correctly classified 111 of the 114 test observations:

$$\text{Test accuracy} = \frac{111}{114} \approx 0.973684 = 97.37\%.$$

It therefore achieved the same test accuracy as the PyTorch logistic model and was about 0.88 percentage points below the scikit-learn baseline.

Results

Model	Correct predictions	Test accuracy
Scikit-learn logistic regression	112/114	98.25%
PyTorch logistic regression	111/114	97.37%
PyTorch probit regression	111/114	97.37%

These results compare the models on one fixed test split. The difference is only one test observation, so it does not establish that one method is generally better.

Code and running instructions

With the Python environment activated, install the required packages and run:

```
python -m pip install numpy scikit-learn torch
python question5_classification.py
```

Question 5: Regression

Data and baseline

I used the Average Localization Error dataset from Question 4. The predictors were `anchor_ratio`, `trans_range`, `node_density`, and `iterations`. The target was `ale`; `sd_ale` was not used.

I split the data into 80% training and 20% test data using `random_state=42`. This produced 85 training observations and 22 test observations. I standardized the predictors using the training-set means and standard deviations, then applied the same transformation to the test set. The target remained in its original units.

For the baseline, I fitted scikit-learn's `LinearRegression` on the training data. I evaluated all models using test mean absolute error:

$$\mathrm{MAE} = \frac{1}{n_{\mathrm{test}}} \sum_{i=1}^{n_{\mathrm{test}}} |y_i - \hat{y}_i|.$$

The baseline test MAE was approximately 0.195393.

(a) Linear regression with squared-error loss

I implemented the linear model

$$f(x_i) = w^\top x_i + b$$

in PyTorch and minimized the mean squared-error loss:

$$\mathcal{L}_{\mathrm{MSE}}(w, b) = \frac{1}{n} \sum_{i=1}^{n} \Big(y_i - w^\top x_i - b \Big)^2.$$

I initialized the weights and bias to zero. I used `torch.optim.SGD` with a learning rate of 0.01, a batch size of 16, and 1,000 epochs. The training observations were shuffled each epoch, and PyTorch computed the gradients automatically.

The test MAE was approximately 0.194155, compared with 0.195393 for the baseline. The results were very similar, as both methods fit a linear model using squared-error loss. The small difference reflects the finite SGD training procedure.

(b) Linear regression with Student-t loss

I used the same linear model but replaced squared-error loss with the negative log-likelihood of a Student-t distribution with $\nu = 5$ and scale fixed at 1, as specified in the question.

Writing $r_i = y_i - f(x_i)$, the loss for one observation is

$$\ell(r_i) = -\log\Gamma(3) + \log\Gamma\left(\frac{5}{2}\right) + \frac{1}{2}\log(5\pi) + 3\log\left(1 + \frac{r_i^2}{5}\right).$$

The training objective was the average of this loss:

$$\mathcal{L}_t(w, b) = \frac{1}{n}\sum_{i=1}^{n} \ell(r_i).$$

I implemented it using `torch.distributions.StudentT` and its `log_prob` method. I used the same initialization and SGD settings as in part (a).

The test MAE was approximately 0.193860. This was slightly lower than both the baseline MAE of 0.195393 and the squared-error model's MAE of 0.194155.

The Student-t loss grows more slowly for large residuals than squared-error loss, reducing their influence on the fit. However, the improvement on this test split was small, so it does not establish that Student-t loss is generally better.

Regression results

Model	Test MAE
Scikit-learn linear regression	0.195393
PyTorch linear regression, MSE	0.194155
PyTorch linear regression, Student-t loss	0.193860

Regression code and running instructions

With the Python environment activated, install the dependencies and run:

```
python -m pip install pandas scikit-learn torch
python question5_regression.py
```